VETRI-THIRAN PAYIRRCHI THITTAM

PROJECT DOCUMENTATION REPORT

PocketSmart AI

Team id:SWTID-2026-5828
Team Size:04
Submitted By:
Team Leader: Ramprasanna M
Team Member: Abimugeshwaran M
Team Member: Sridhar R
Team Member: Saravanan G

Vetri-Thiran Payirrchi Thittam is a web-based learning and training platform intended to help students and learners access courses, practice questions, take assessments, track performance, and obtain assistance through a Question & Answer system. An administrative interface provides centralized control over users, courses, questions, tests, reports and application settings.

Project Objectives

- Create a centralized platform for learning, practice and assessment.
- Provide role-based access for administrators, trainers and students.
- Allow administrators to manage users, courses, questions and tests.
- Provide a searchable Question & Answer experience.
- Track learner activity, scores and progress through reports and dashboards.
- Provide an architecture that can support an AI/LLM service for intelligent educational assistance.

1. Major Use Scenarios

Scenario 1: Admin Dashboard

An administrator signs in and reaches the Admin Dashboard. The dashboard presents key metrics, user growth, user roles, course performance, recent users and quick actions. This gives the administrator a single view of platform activity.

Scenario 2: Question & Answer

A learner opens Question & Answer, searches existing questions, filters by category, reviews recent questions, or selects Ask a Question. Each question shows its category and answer status. The design also includes recent activity and a helpful-rate indicator.

Scenario 3: Courses and Assessment

Students access courses and training content, complete practice questions or tests, and receive recorded scores. Trainers and administrators can manage learning and assessment content and review results.

2. Architecture Overview

The system is designed as a modular full-stack application. The frontend handles presentation and user interaction. The backend provides authentication, business logic, API routes and data access. A database stores platform information. An optional AI service can be connected to the Q&A workflow.

Component	Description
Frontend UI	Responsive web interface for dashboards, courses, questions, tests, reports and settings.
Backend API	Authentication, authorization, validation, CRUD operations and business logic.
Database	Stores users, roles, courses, questions, answers, tests, attempts and activity.
AI/LLM Layer	Optional service for AI-assisted educational answers and explanations.
Authentication	Login, session/token handling and role-based access control.
Reporting	Aggregates activity, enrollment, test and score information for analytics.

Core Technologies

- Frontend: HTML, CSS and JavaScript or a modern frontend framework.
- Backend: Python with FastAPI or an equivalent REST API framework.
- Database: SQLite for development and PostgreSQL or another relational database for deployment.
- AI integration: An LLM API connected through a dedicated backend service.
- Development: Visual Studio Code and Git/GitHub.

3. Pre-requisites

1. Python 3.x.
2. Visual Studio Code.
3. Git and a GitHub account if source control is required.
4. Python virtual environment.
5. Project dependencies from requirements.txt.
6. Environment configuration through a .env file.
7. LLM API key if AI-assisted Q&A is enabled.
8. Development or production database.

4. Project Workflow

4.1 Authentication & Access Control

- Register and authenticate users.
- Protect private routes.
- Assign roles such as Admin, Trainer and Student.
- Redirect users to role-appropriate pages.

4.2 Admin Management

- Manage users and roles.
- Create, edit and remove courses.
- Manage questions and tests.
- Review reports and recent activity.
- Maintain system settings.

4.3 Learning & Training

- Display available courses.
- Provide learning material.
- Provide practice questions and assessments.
- Record attempts, scores and progress.

4.4 Question & Answer

- Search existing questions.
- Filter questions by category.
- Submit new questions.
- Display answered and pending questions.
- Optionally generate AI-assisted answers.

4.5 Testing & Optimization

- Test functional workflows.
- Validate API responses and input.
- Measure dashboard and Q&A performance.
- Improve prompts, validation and error handling.

5. Development Milestones

MILESTONE 1: Project Initialization

Set up the repository, backend, frontend, database and configuration.

- Create the project structure.
- Create a virtual environment.
- Install dependencies.
- Create .env.example and configuration handling.
- Initialize database models.

MILESTONE 2: Authentication & Roles

Implement secure access and role-based permissions.

- Registration and login.
- Password hashing.
- JWT/session support.
- Admin, Trainer and Student authorization.

MILESTONE 3: Admin Dashboard

Implement the administrative overview represented by the supplied UI concept.

- Metric cards.
- User growth chart.
- User-role chart.
- Top-performing courses table.
- Recent users table.
- Quick Actions.

MILESTONE 4: Question & Answer

Implement the supplied Q&A page concept.

- Search bar.
- Ask a Question workflow.
- Question statistics.
- Category list.
- Recent Questions list.

- Answered/Pending states.
- Recent Activity.

MILESTONE 5: Courses, Tests & Reports

Implement core learning and assessment management.

- Course CRUD.
- Question bank.
- Test creation.
- Test attempts.
- Score calculation.
- Progress reports.

MILESTONE 6: AI Integration

Connect an LLM service to the Q&A workflow.

- Secure API-key configuration.
- AI service layer.
- Educational Q&A prompts.
- Response validation and formatting.
- Fallback response when AI is unavailable.

MILESTONE 7: Testing & Deployment

Validate the application and prepare it for deployment.

- Functional testing.
- API testing.
- Role/access testing.
- Responsive UI testing.
- Error handling.
- Production configuration.

6. Backend API Design

Endpoint	Method	Purpose	Access
/auth/register	POST	Create account	Public
/auth/login	POST	Authenticate user	Public
/auth/logout	POST	End session	Authenticated
/users	GET/POST	List/create users	Admin
/users/{id}	GET/PUT/DELETE	Manage a user	Admin
/courses	GET/POST	List/create courses	Authenticated/Admin
/courses/{id}	GET/PUT/DELETE	Manage course	Admin
/questions	GET/POST	Browse/submit questions	Authenticated
/questions/{id}	GET/PUT/DELETE	Manage question	Owner/Admin
/tests	GET/POST	Manage tests	Admin/Trainer
/tests/{id}/submit	POST	Submit test attempt	Student

/reports/dashboard	GET	Dashboard metrics	Admin
/qa/search	GET	Search questions	Authenticated
/qa/ask	POST	Submit Q&A request	Authenticated

7. Data Model

Entity	Important Fields	Purpose
User	id, name, email, password_hash, role, status	Platform users and access roles.
Course	id, title, description, category, status	Learning courses.
Question	id, user_id, category, title, body, status	Learner questions.
Answer	id, question_id, user_id, body, source	Answers and optional AI responses.
Test	id, course_id, title, duration, status	Assessments.
TestQuestion	id, test_id, question_id, marks	Maps questions to tests.
Attempt	id, user_id, test_id, score, submitted_at	Test attempts and results.
Activity	id, user_id, action, timestamp	Recent activity and reporting.

8. User Interface Documentation

8.1 Admin Dashboard

The Admin Dashboard uses a left navigation sidebar and a main content workspace. The visual concept contains Dashboard, Users, Courses, Questions & Tests, AI Chat, Reports, Settings and Logout navigation.

- Total Users metric.
- Total Courses metric.
- Total Tests metric.
- Average Score metric.
- User Growth line chart.
- User Roles distribution chart.
- Top Performing Courses table.
- Recent Users table.
- Quick Actions for adding users, courses and questions and viewing reports.



Figure 1. Sample Admin Dashboard screen

8.2 Question & Answer Page

The Question & Answer page is the learner-facing knowledge and assistance area. It combines search, question creation, statistics, category navigation, recent questions and recent activity in one workspace.

- Question & Answer page heading and description.
- Search questions field.
- Ask a Question button.
- Total Questions, Answered, Pending and Helpful Rate cards.
- Recent Questions with user, time, category and status.
- Question Categories list with category counts.
- Recent Activity list.

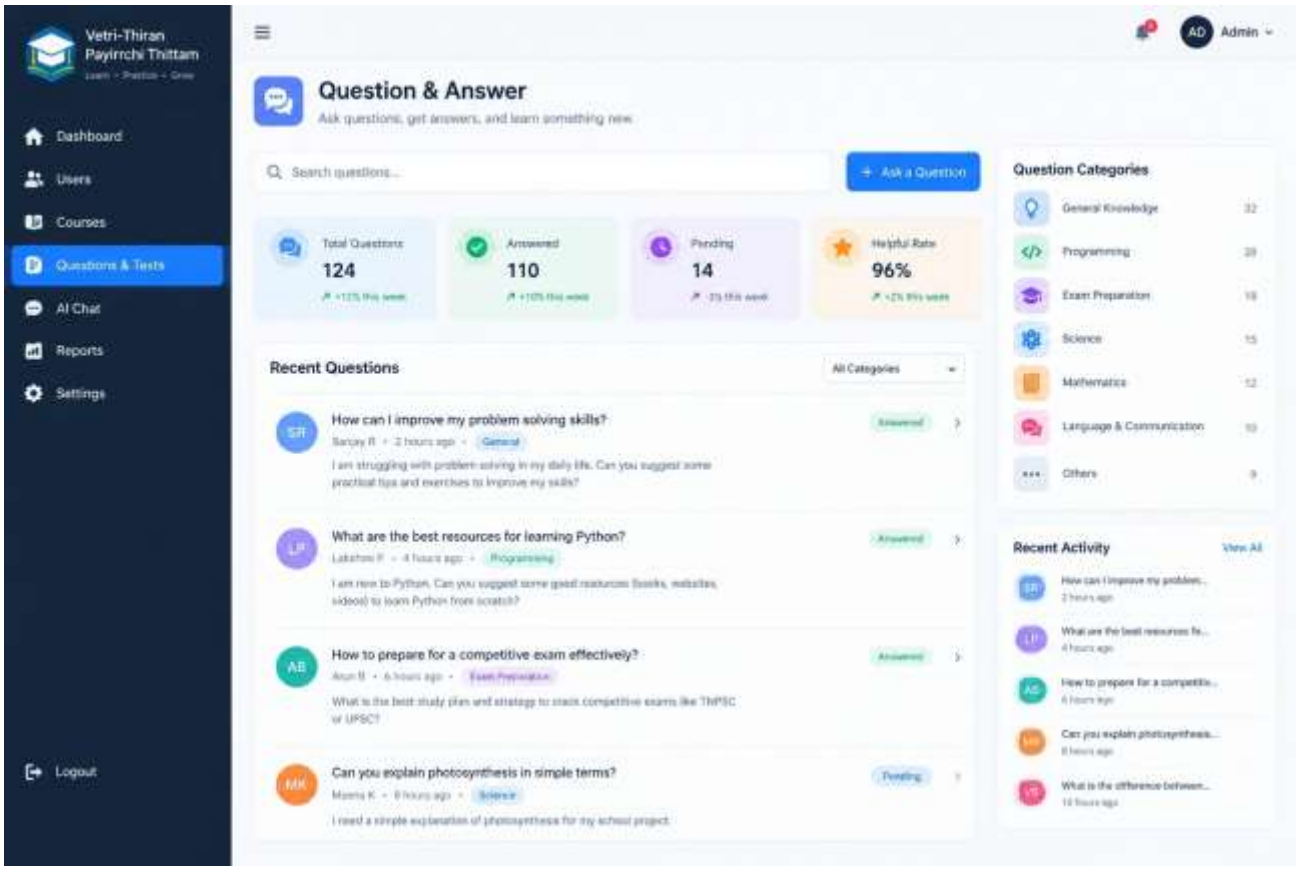


Figure 2. Sample Question & Answer screen

9. Security & Validation

- Store passwords only as secure hashes.
- Validate authentication and authorization on the server.
- Restrict administrative operations to authorized roles.
- Keep API keys and secrets in environment variables.
- Validate all user-submitted data.
- Use ORM/parameterized database operations.
- Do not expose secrets or unnecessary personal data to an AI model.
- Consider rate limiting for public-facing AI endpoints.

10. Testing & Optimization

Test Area	Example	Expected Result
Login	Valid/invalid credentials	Valid login succeeds; invalid login returns safe error.
Authorization	Student opens admin route	Access is denied.
Users	Admin creates user	User appears in list.
Courses	Admin edits course	Updated course is displayed.
Q&A	User submits question	Question is saved with status.
Search	Keyword/category search	Matching questions are returned.
AI Q&A	Submit educational question	Answer is returned or fallback is shown.

Tests	Student submits test	Attempt and score are stored.
Dashboard	Open admin dashboard	Metrics and charts render.
Responsive UI	Different screen sizes	Layout remains readable and usable.

11. Installation, Running & Testing

9. Open the project folder in Visual Studio Code.
10. Create and activate a Python virtual environment.
11. Install backend dependencies from requirements.txt.
12. Create .env from .env.example.
13. Configure the database and AI API key if AI is enabled.
14. Start the backend using the project's ASGI server command.
15. Start the frontend development server if the project uses one.
16. Create/seed an administrator account for development.
17. Open the Admin Dashboard and verify metrics, tables and navigation.
18. Open Question & Answer and test search and Ask a Question.
19. Run API and functional tests before deployment.

12. Recommended Project Folder Structure

```
Vetri-Thiran-Payirrchi-Thittam/
├── backend/
│   ├── app/
│   │   ├── main.py
│   │   ├── config.py
│   │   ├── database.py
│   │   ├── models/
│   │   ├── schemas/
│   │   ├── routes/
│   │   ├── services/
│   │   │   └── ai_service.py
│   │   └── utils/
│   ├── tests/
│   ├── requirements.txt
│   └── .env.example
├── frontend/
│   ├── index.html
│   ├── pages/
│   ├── css/
│   ├── js/
│   └── assets/
├── README.md
└── .gitignore
```

13. Future Enhancements

- AI-generated explanations with different difficulty levels.

- Tamil and English language support.
- Personalized learning recommendations based on performance.
- Voice-based Question & Answer.
- Mobile/PWA support.
- Advanced trainer and administrator analytics.
- Notifications for new courses, tests and unanswered questions.
- Gamification with points, badges and leaderboards.
- Exportable student/course reports.
- Production monitoring and scalable deployment.

14. Conclusion

Vetri-Thiran Payirrchi Thittam is designed as a practical learning and training platform that brings together structured learning, assessment, reporting and intelligent Question & Answer support. The Admin Dashboard provides centralized management and analytics, while the Q&A interface provides a clear workflow for finding and asking questions.

The architecture can be developed incrementally: authentication and database first, followed by courses and assessments, dashboards and Q&A, and finally AI-assisted educational features. The two supplied UI concepts serve as the visual reference for the administrative and Q&A modules.

15. Documentation Basis

The uploaded PocketSmart AI document was used as the sample format. Its organization around project description, scenarios, architecture, prerequisites, workflow, milestones, API/backend work, UI development, testing and conclusion was adapted for this project. The content here describes the Vetri-Thiran Payirrchi Thittam concept and the supplied dashboard/Q&A designs rather than reproducing the PocketSmart functionality.